

Object Oriented Programming

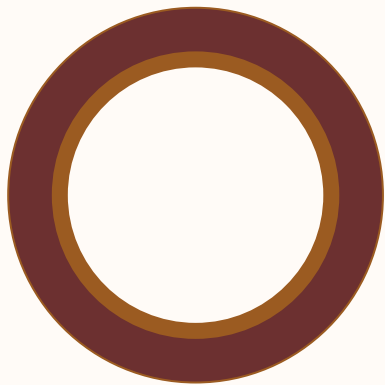
CS 22B, Spring 2026

Module 05: Data Exploration

Presented by Jessica Huynh-Westfall, Ph.D.

AGENDA

- What is OOP?
- Object, Class and Instances



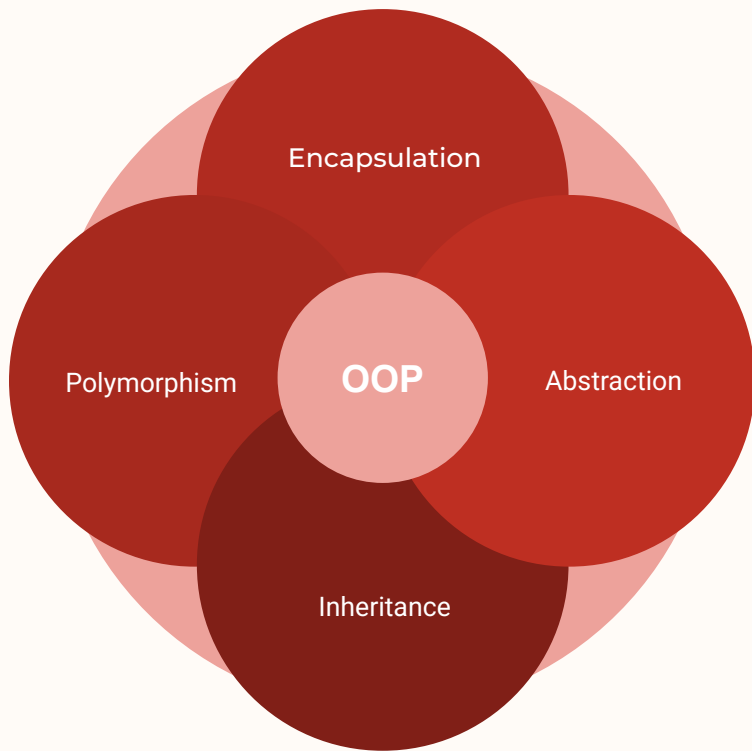
Chapter 15: Classes and object-oriented programming

Quick Python



What is object-oriented programming?

04



OOP allows us to *organize* code around “objects”. This enable us to model any data (aka object) that we need and to work with the data in a logical way

Core principles of OOP

- Encapsulation - Bundling data
- Abstraction - Show essential features
- Inheritance - child can inherit properties and method from parent
- Polymorphism - class can take many form

What is an Object?

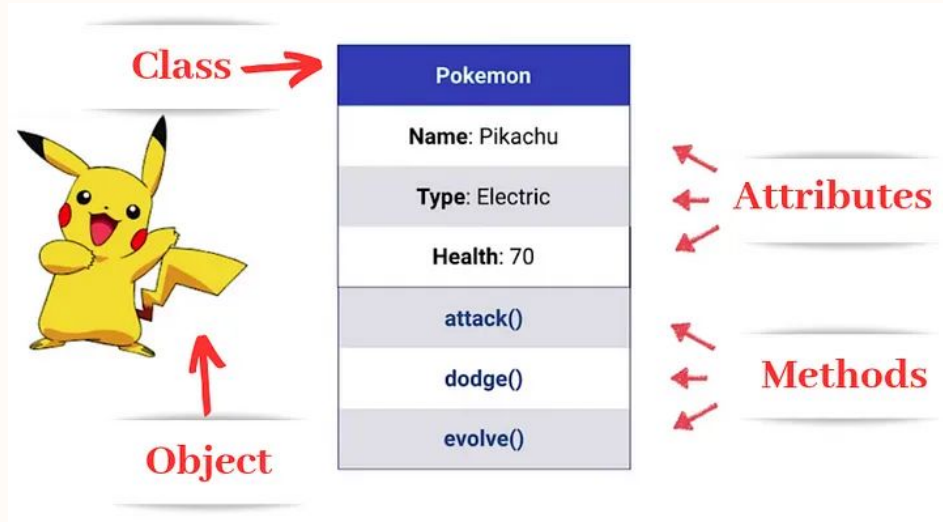
05

Objects are real world entities.

They contain:

- **attributes** (Field of data)
- **methods** (Functions)

Class is the blueprint for objects



Objects can be:

- Something visible for you, like the car, phone, or book
- Something that you cannot touch, like the time, event, or account

Class exercise

06

Scenario:

You want to create a class for Car. Think about what do we know about cars and work with a partner to think of the different attributes and methods that a car may have

*What do we know about **Cars**?*

Objects have **Attributes**

07

- They are *characteristics* or *properties* of an object
- They describe the *current state* of an object
- The state of one object is independent of another
 - eg., Honda Accord vs Ford Fusion



Objects have Methods

08

- Anything that the object can do (function)
- Examples:
 - A car can START
 - We can DEPOSIT or/and WITHDRAW form a bank account
 - A dog can BARK



How do you construct an object?

09

- The attributes (properties) and the methods (behaviors) of an **Object** are defined using a **Class**
- A **Class** is a blueprint for creating an **object**



What is a Class?

The attributes (properties) and the methods (behaviors) of an object are defined using a **Class**

Car
model : String color : String
start() stop() description()

Car class

```
class Car:
    ## Initializer method - set up the attributes for
    Car class
    def __init__(self, make, model, color):
        self.make = make
        self.model = model
        self.color = color

    def get_description(self):
        return f"The {self.make} {self.model} is a
        {self.color} color."
```

What is a object Instance?

An **Instance** is the copy of the class with actual values (a particular object)

Car
model : String color : String
start() stop() description()

Car class

Car object	
model	Hon
color	Blu
start() stop() description()	

A Car object

Car object	
model	BMW i3
color	Silver
start() stop() description()	

A Car object

```
## Instantiate a Car object  
toyota_camry = Car("Toyota", "Camry", "blue")  
honda_civic = Car("Honda", "Civic", "silver")  
  
## Call our instance method to get the description of  
our cars  
print(toyota_camry.get_description())  
print(honda_civic.get_description())
```

Differentiating Object or Instances

12

Almost the same thing, but . . .

- **Class:** Blueprint / Template
 - Eg., Blueprint for building Car.. What make is it? What is its color? No real, physical car at this point
- **Instance:** A unique copy of the blueprint. Each **Instance** holds different data.
 - Eg., Each Car is an **Instance** and is unique. A blue Car and a silver Car follows the same blueprint
- **Object:** Physical Presence of the **Class** in the memory
 - Eg., Once Car is built (an **Instance**), it physically exist.